

INFORMAÇÕES

Módulo: 09

Aula: 07 - Context e Provider

Instrutor: Lucca Dumas

Duração: 2 h

CONTEÚDO

1) Um pulinho na aula anterior

Na **Aula 06**, vimos duas abordagens de estado compartilhado no projeto:

- Context + Provider no projeto (ex.: toasts);
- Zustand para estado global em pontos do sistema (ex.: empresa ativa).

Nesta aula, o foco é consolidar **Context + Provider** com um exemplo simples de compartilhamento de dados entre **páginas diferentes**.

2) O que veremos hoje?

1. Revisão de Context/Provider
 2. Exemplo simples compartilhando dados entre páginas
 3. Diferença entre props, estado local e contexto global da aplicação
 4. Hands-on com duas páginas lendo/escrivendo o mesmo contexto
 5. Ponte para `useCallback` e `useMemo` (próxima aula)
-

3) Revisão rápida: Context/Provider

Objetivo da revisão: reforçar a mecânica base de compartilhamento de dados sem prop drilling.

Estrutura essencial

1. Criar contexto (`createContext`)
2. Expor `Provider` com estado (`useState`)
3. Consumir via hook (`useContext`)

Exemplo base

```
import { createContext, useContext, useState, type ReactNode } from "react";

type MensagemContextType = {
  mensagem: string;
  definirMensagem: (texto: string) => void;
  limparMensagem: () => void;
};

const MensagemContext = createContext<MensagemContextType | undefined>(undefined);

export function MensagemProvider({ children }: { children: ReactNode }) {
  const [mensagem, setMensagem] = useState("");

  return (
    <MensagemContext.Provider
      value={{
        mensagem,
        definirMensagem: (texto) => setMensagem(texto),
        limparMensagem: () => setMensagem(""),
      }}
    >
      {children}
    </MensagemContext.Provider>
  );
}

export function useMensagem() {
  const ctx = useContext(MensagemContext);
  if (!ctx) {
    throw new Error("useMensagem deve ser usado dentro de MensagemProvider");
  }
  return ctx;
}
```

4) Exemplo principal no projeto de treino

Objetivo: comparar as duas abordagens em páginas separadas, com a mesma ideia de dado (`mensagem`):

- [Página Context/Provider](#)
- [Página Zustand](#)

Estrutura atual do exemplo

```
src/
  App.tsx
  components/
    Button.tsx
  contexts/
    mensagemContext.ts
    MensagemProvider.tsx
  hooks/
    useMensagem.ts
  pages/
    PaginaContexto.tsx
    PaginaZustand.tsx
  store/
    mensagemZustandStore.ts
    useMensagemZustandStore.ts
```

Código do fluxo Context/Provider

`mensagemContext.ts` (contrato do contexto):

```
import { createContext } from "react";

export type MensagemContextType = {
  mensagem: string;
  definirMensagem: (texto: string) => void;
  limparMensagem: () => void;
};

export const MensagemContext = createContext<MensagemContextType | undefined>(
  undefined,
);
```

MensagemProvider.tsx (estado + Provider):

```
import { useState, type ReactNode } from "react";
import { MensagemContext } from "../mensagemContext";

export function MensagemProvider({ children }: { children: ReactNode }) {
  const [mensagem, setMensagem] = useState("");

  return (
    <MensagemContext.Provider
      value={{
        mensagem,
        definirMensagem: (texto) => setMensagem(texto),
        limparMensagem: () => setMensagem(""),
      }}
    >
      {children}
    </MensagemContext.Provider>
  );
}
```

useMensagem.ts (hook de consumo com validação):

```
import { useContext } from "react";
import { MensagemContext } from "../contexts/mensagemContext";

export function useMensagem() {
  const ctx = useContext(MensagemContext);
  if (!ctx) {
    throw new Error("useMensagem deve ser usado dentro de MensagemProvider");
  }
  return ctx;
}
```

PaginaContexto.tsx (página que usa Context):

```
import Button from "../components/Button";
import { useMensagem } from "../hooks/useMensagem";

export function PaginaContexto() {
  const { mensagem, definirMensagem, limparMensagem } = useMensagem();

  return (
    <section className="page-card">
      <h2>Pagina Context/Provider</h2>
      <p><strong>Mensagem atual:</strong> {mensagem || "Nenhuma"}</p>
      <div className="actions">
        <Button onClick={() => definirMensagem("Mensagem definida pela pagina Context")}>
          Definir mensagem
        </Button>
        <Button onClick={limparMensagem}>Limpar</Button>
      </div>
    </section>
  );
}
```

Código do fluxo Zustand

mensagemZustandStore.ts:

```
import { createStore } from "zustand/vanilla";

export type MensagemZustandState = {
  mensagem: string;
  definirMensagem: (texto: string) => void;
  limparMensagem: () => void;
};

export const mensagemZustandStore = createStore<MensagemZustandState>((set) => ({
  mensagem: "",
  definirMensagem: (texto) => set({ mensagem: texto }),
  limparMensagem: () => set({ mensagem: "" }),
}));
```

useMensagemZustandStore.ts:

```
import { useStore } from "zustand";
import { mensagemZustandStore } from "../mensagemZustandStore";

export function useMensagemZustandStore() {
  return useStore(mensagemZustandStore);
}
```

PaginaZustand.tsx:

```
import Button from "../components/Button";
import { useMensagemZustandStore } from "../store/useMensagemZustandStore";

export function PaginaZustand() {
  const { mensagem, definirMensagem, limparMensagem } = useMensagemZustandStore();

  return (
    <section className="page-card">
      <h2>Pagina Zustand</h2>
      <p><strong>Mensagem atual:</strong> {mensagem || "Nenhuma"}</p>
      <div className="actions">
        <Button onClick={() => definirMensagem("Mensagem definida pela pagina Zustand")}>
          Definir mensagem
        </Button>
        <Button onClick={limparMensagem}>Limpar</Button>
      </div>
    </section>
  );
}
```

App.tsx (montagem das duas páginas)

```
import { MensagemProvider } from "../contexts/MensagemProvider";
import { PaginaContexto } from "../pages/PaginaContexto";
import { PaginaZustand } from "../pages/PaginaZustand";

function PaginaComContexto() {
  return (
    <MensagemProvider>
      <PaginaContexto />
    </MensagemProvider>
  );
}

export default function App() {
  return (
    <main className="app-shell">
      <PaginaComContexto />
      <PaginaZustand />
    </main>
  );
}
```

Explicando por fases (passo a passo)

Fase 1 — Definição do contrato

Definimos que a mensagem tem três operações: ler (`mensagem`), escrever (`definirMensagem`) e limpar (`limparMensagem`).

Fase 2 — Origem do estado

- No Context: estado nasce no `MensagemProvider` com `useState` .
- No Zustand: estado nasce na store com `createStore` .

Fase 3 — Ponto de acesso

- No Context: consumo via `useMensagem()` (com validação de Provider).
- No Zustand: consumo via `useMensagemZustandStore()` .

Fase 4 — Interação na página

As duas páginas chamam os mesmos tipos de ação (`definirMensagem` e `limparMensagem`) para facilitar comparação.

Fase 5 — Resultado visível

Cada página exibe sua própria mensagem atual, permitindo observar o fluxo completo de leitura/escrita em cada abordagem.

5) Props x estado local x Context

Caso	Melhor escolha
Dado usado por 1 componente	<code>useState</code> local
Dado passado pai -> filho direto	<code>props</code>
Dado compartilhado por páginas/níveis diferentes	Context + Provider (ou store, conforme o projeto)

Regra prática:

- primeiro avaliar estado local;
- se o dado precisa atravessar vários níveis/páginas, avaliar Context;
- seguir o padrão já adotado no projeto quando houver store global.

6) Hands-on da aula

Objetivo do hands-on

1. Montar a página de Context (`PaginaContexto`) com `MensagemProvider` e `useMensagem` ;
2. Montar a página de Zustand (`PaginaZustand`) com `mensagemZustandStore` e `useMensagemZustandStore` ;
3. Exibir as duas no `App.tsx` ;
4. Validar o fluxo com `yarn lint` e execução local.

Passos

1. Criar `mensagemContext.ts` , `MensagemProvider.tsx` e `useMensagem.ts` .
2. Criar `mensagemZustandStore.ts` e `useMensagemZustandStore.ts` .
3. Criar `PaginaContexto.tsx` e `PaginaZustand.tsx` .
4. No `App.tsx` , envolver apenas `PaginaContexto` com `MensagemProvider` .
5. Executar o projeto e validar os botões de definir/limpar nas duas páginas.
6. Rodar `yarn lint` .

7) Erros comuns desta aula

1. **Usar hook fora do Provider**
Resultado: erro em runtime.
2. **Não envolver todas as páginas necessárias no Provider**
Resultado: parte da árvore não acessa o contexto.
3. **Criar contexto sem tipagem clara**
Resultado: consumo confuso e maior chance de bug.
4. **Tentar resolver tudo com Context**
Para estado local, `useState` geralmente é a melhor escolha.

8) Exercícios de fixação

1. Context + Provider ajuda principalmente quando:
☐ Só um componente usa o dado
☒ O mesmo dado precisa ser usado em páginas/componentes diferentes
☐ O projeto não usa TypeScript
☐ Queremos substituir rotas
2. Se `PaginaResumo` usar `useMensagem` fora do Provider:
☐ O React ignora
☒ Ocorre erro em runtime
☐ Apenas o TypeScript reclama
☐ Nada acontece
3. Para um dado usado apenas no componente atual, o mais comum é:
☒ `useState` local
☐ Criar Context sempre
☐ Criar store global sempre
☐ Passar por rota
4. Qual comando roda lint no projeto com Yarn?
☐ `yarn dev`
☒ `yarn lint`

- ☐ yarn preview
- ☐ yarn build

5. No exemplo de duas páginas, qual ação muda o valor lido na página de resumo?
- ☐ Trocar a importação do arquivo
 - ☒ Chamar `definirMensagem(...)` na página Home
 - ☐ Criar novo componente Button
 - ☐ Reiniciar o Vite

9) Desafio para casa

1. Criar `UsuarioContext` com:

- `nomeUsuario: string;`
- `definirNome(nome: string);`
- `limparNome();`

2. Criar duas páginas:

- `PaginaPerfil` (define nome);
- `PaginaDashboard` (exibe nome).

3. Envolver as páginas com `UsuarioProvider`.

4. Rodar:

```
yarn lint
```

5. Escrever em 5 linhas:

- quando usar estado local;
- quando usar props;
- quando usar Context + Provider;
- onde o projeto usa Zustand;
- qual foi o fluxo de atualização entre as duas páginas.

Regras

- manter nomes claros;
- tipar contexto e Provider;
- manter validação do hook fora do Provider.

10) Ponte para a próxima aula

Na próxima aula, avançaremos em otimização com:

- `useCallback` (memoização de função);
- `useMemo` (memoização de valor calculado).

Nesta aula, a menção a esses hooks fica apenas como preparação.

11) Resumo final

Ao final desta aula, o aluno deve conseguir:

- criar e consumir um Context com Provider;
- compartilhar dados entre páginas/componentes diferentes;
- explicar `props` x `useState` local x Context;
- validar o fluxo com `yarn lint`;
- reconhecer quando Context é adequado no projeto.